SFAMILY · API V1

Chat Integration Guide

Everything the web client needs to reach parity with the mobile app: every endpoint, the websocket handshake in full, and the handful of client-side rules that are load-bearing rather than stylistic.

REST BASE	WEBSOCKET	PROTOCOL
https://admin-cp.sfamily.co/api/v1	wss://ws.sfamily.co:443	Pusher 7 (Laravel Reverb)
AUTH		
Bearer <sanctum token>		

Endpoints and auth

Every chat route sits behind `auth:sanctum`. Sign in once, keep the token, send it as a bearer header on every request including the websocket authorisation call.

```
# Sign in. The token is what everything else uses.
curl -X POST https://admin-cp.sfamily.co/api/v1/auth/login \
  -H 'Content-Type: application/json' \
  -H 'Accept: application/json' \
  -d '{
    "phone_country_code": "+91",
    "phone_number": "9876543210",
    "password": "secret123"
  }'

# → data.token, data.user  (user.id is a uuid)
```

The envelope

Every response — success or failure — has the same four keys. The web client should unwrap this in one place and never parse a raw body anywhere else.

```
{
  "success": true,
  "message": "OK",
  "data": { /* the payload, or null */ },
  "errors": null       // on 422: { "field": ["message"] }
}
```

> ONE EXCEPTION
>
> `POST /broadcasting/auth` is Laravel's own broadcasting endpoint and returns the object directly — no envelope. It is the only route in the API that does.

Conventions

UUIDs, never numbers

Auto-increment ids never leave the server. Every `id` in every payload — users, conversations, messages, attachments — is a uuid, including the identifier inside presence-channel membership. If you ever see an integer id in a response, that is a bug worth reporting rather than working around.

seq is the ordering key

Each message carries a `seq`: an integer starting at 1, unique and gapless *within its conversation*, assigned under a row lock at insert. It is the sort key, the pagination cursor and the unit read receipts are measured in. Never sort by `created_at` — two messages can share a timestamp, and a message that overtakes another in flight must still land in the right place.

`client_uuid` makes retries safe

The client generates a v4 uuid before a send leaves the device and keeps it across retries. The server has a unique index on `(conversation_id, client_uuid)`: sending the same one twice returns the original message with `200` rather than creating a second one or failing with a conflict. From the sender's point of view the message was sent, which is true.

It is also how the optimistic bubble is reconciled. Messages broadcast over the socket carry `client_id`, so the sender's own echo resolves to the bubble already on screen instead of appearing twice.

Pagination

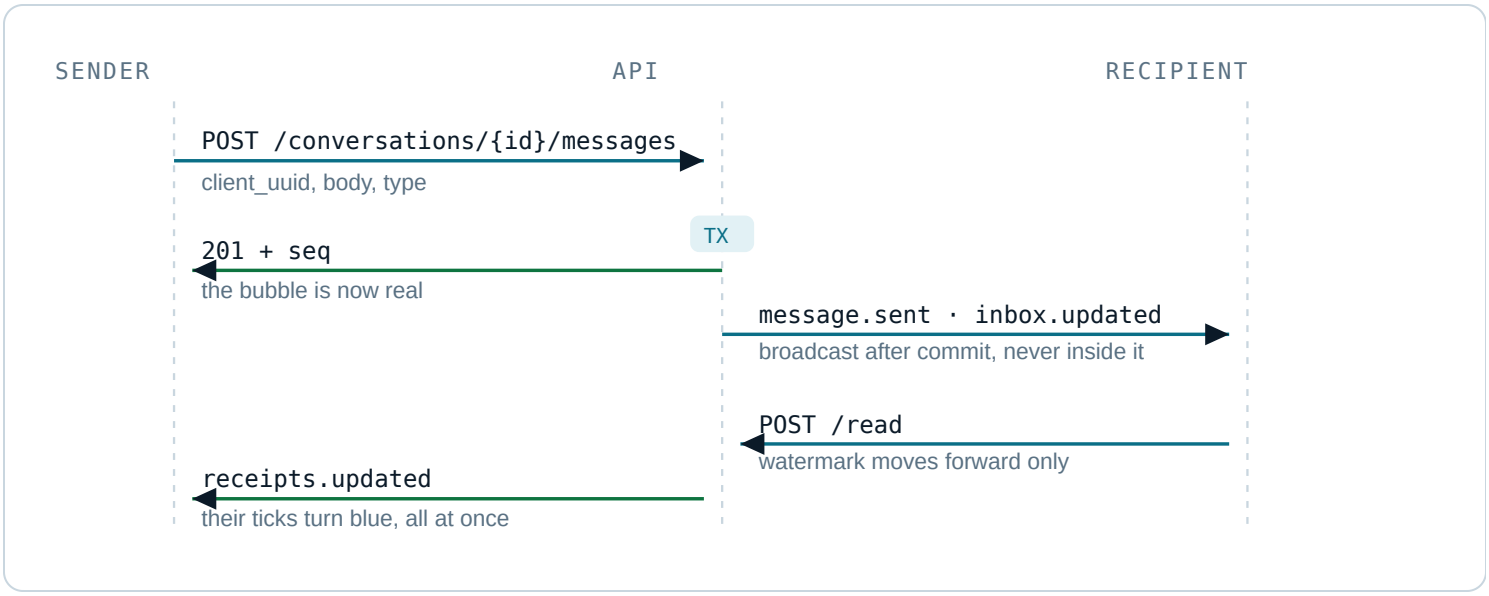
Listings return `{ total, page, per_page, has_more, … }`. Message history is different — it is cursor-based on `seq`, because a page number over a list that grows from the bottom is a race condition waiting to happen.

Rate limits

ROUTE	LIMIT	WHY
`POST messages`	60/min	A person typing, not a script
`read / delivered`	240/min	Fired on every scroll tick; both are idempotent no-ops when the watermark is already ahead
`react / star / hide`	120/min	Toggles
`forward`	30/min	Ten targets each
`clear`	20/min	Writes a row per message in the thread
`uploads`	30/min	Bytes

Message lifecycle

The rule the whole design rests on: **messages travel over HTTP; the socket only announces that one arrived.** Nothing is ever only in the socket. A send is a transaction that either commits or does not, is safe to retry, and works with the socket down.



Events are dispatched *after* the database transaction returns. An event that arrives before its own row is visible is a bug that only shows up under load.

> NO TOOTHERS()
>
> The sender receives their own `message.sent` frame too. That is deliberate — it is what makes multi-device support a change of mind rather than a rewrite. Deduplicate on `client_id`, not by filtering on sender.

Conversations

`GET` `/conversations`

the inbox

QUERY	DEFAULT	MEANING
`state`	`accepted`	`pending` is the Requests tab
`archived`	—	`1` returns the Archived list instead
`page`	1	25 per page

```
curl -H 'Authorization: Bearer $TOKEN' \
  'https://admin-cp.sfamily.co/api/v1/conversations?state=accepted&page=1'
```

RESPONSE · DATA

```
{
  "total": 14, "page": 1, "per_page": 25, "has_more": false,
  "conversations": [{
    "id": "9d2f…",
    "type": "direct",
    "state": "accepted",        // pending = a request
    "unread_count": 3,
    "muted": false,
    "muted_until": null,
    "pinned": false,           // pinned chat — mine alone
    "marked_unread": false,
    "archived": false,
    "blocked": false,          // a wall, in either direction
    "last_message_at": "2026-09-01T18:22:04+00:00",
    "last_message": { /* a message object */ },
    "pinned_message": null,    // pinned message — shared by both
    "me":    { "last_read_seq": 40, "last_delivered_seq": 40 },
    "other": {
      "id": "7a10…", "name": "Aisha", "username": "aisha",
      "avatar_url": "https://…",
      "state": "accepted",
      "last_read_seq": 38, "last_delivered_seq": 40,
      "online": true,            // null = hidden by their setting
      "last_seen_at": "2026-09-01T18:20:00+00:00"
    }
  }]
}
```

> ORDERING
>
> Pinned chats first, then newest. Repeat that order client-side when you fold a socket event into the list — if the two disagree, a pinned chat drops down the list until the next refresh puts it back.

GET `/conversations/unread-count`

badges

POST `/conversations`

open a thread

`GET` `/conversations/{uuid}`

one thread

`POST` `/conversations/{uuid}/accept`

accept a request

`DELETE` `/conversations/{uuid}`

delete chat / decline

Messages

`GET` `/conversations/{uuid}/messages`

scrollback

QUERY	USE
`before`	Walk backwards as the user scrolls up. Returns the page ending just before that seq.
`after`	Everything since a seq you already hold — how a screen recovers what it missed while the socket was down.
`limit`	Default 40, max 100.

```
# newest page
curl -H 'Authorization: Bearer $TOKEN' \
  '…/conversations/$CID/messages?limit=40'

# older, scrolling up
curl … '…/messages?before=41&limit=40'

# the gap after a reconnect
curl … '…/messages?after=80'
```

RESPONSE · DATA

```
{
  "messages": [ /* ascending by seq */ ],
  "has_more": true,     // only meaningful on the before= path
  "oldest_seq": 41,
  "newest_seq": 80
}
```

THE MESSAGE OBJECT

```
{
  "id": "3c81…",              // server uuid
  "client_id": "b0f2…",       // the sender's client_uuid — dedupe on this
  "seq": 42,
  "sender_id": "7a10…",
  "type": "text",             // text | image | file | audio | system
  "body": "see you saturday",
  "deleted": false,
  "forwarded": false,
  "starred": false,           // yours; never visible to them
  "attachment": null,
  "reply_to": {
    "id": "2b70…", "sender_id": "9f31…",
    "body": "what time?", "type": "text", "deleted": false
  },
  "reactions": [
    { "emoji": "❤️", "count": 2, "user_ids": ["7a10…", "9f31…"] }
  ],
  "edited_at": null,
  "created_at": "2026-09-01T18:22:04+00:00"
}
```

A deleted message keeps its row and its place: `deleted` is true, `body` is null, `type` reverts to text and the attachment is withheld. Render a tombstone. A line vanishing out of the middle of a conversation reads as a bug.

`POST` `/conversations/{uuid}/messages`

send

```
curl -X POST …/conversations/$CID/messages \
  -H 'Authorization: Bearer $TOKEN' \
  -H 'Content-Type: application/json' \
  -d '{
    "client_uuid": "b0f2c1de-4a55-4c2f-9d0e-2f4b8a1c7e33",
    "type": "text",
    "body": "see you saturday",
    "reply_to_id": null
  }'
```

FIELD	RULE
`client_uuid`	Required, v4 uuid. The idempotency key.
`type`	`text` (default), `image`, `file`, `audio`
`body`	Required for text; the caption otherwise. Max 4000.
`upload_id`	Required for anything but text — see Uploads.
`reply_to_id`	Optional message uuid, in the same thread.

Returns `201` with the message object. A replayed `client_uuid` returns `200` and the original message — treat both as success.

`DELETE` `/messages/{uuid}`

delete for everyone

`POST` `/messages/{uuid}/hide`

delete for me

Receipts and ticks

Nothing is stored per message. Each participant carries two integers — the highest seq handed to them, and the highest they have read — and every tick in the thread is derived from those on the client.

STATE	CONDITION
Sending	Local only; no server response yet
Sent · one tick	The send returned
Delivered · two grey	`other.last_delivered_seq >= message.seq`
Read · two blue	`other.last_read_seq >= message.seq`

That is what makes a whole column turn blue at once when somebody opens a thread, instead of rippling upward one bubble at a time.

`POST` `/conversations/{uuid}/read`

I have seen up to here

`POST` `/conversations/{uuid}/delivered`

it reached my device

`GET` `/messages/{uuid}/info`

when it was read

> READ RECEIPTS ARE A SETTING
>
> Somebody who has turned them off has their read watermark reported as their delivered one. You will see two grey ticks and never a blue pair, both on a cold load and over the socket. Do not try to detect or work around this.

Message actions

`POST` `/messages/{uuid}/react`

emoji

`POST` `/messages/{uuid}/star`

keep · private

`GET` `/starred-messages`

everything kept

`POST` `/messages/{uuid}/forward`

up to 10 chats

`POST` `/conversations/{uuid}/pin`

pin a message · shared

Thread settings

All five are private to the caller: each writes one column on your own participant row, nothing is broadcast, and the other person cannot tell.

ROUTE	BODY	RETURNS	NOTES
`POST …/pin-chat`	—	`{ pinned }`	Toggles. Sorts to the top of your inbox.
`POST …/archive`	—	`{ archived }`	Toggles, and un-pins. A new message does *not* pull it back out.
`POST …/mute`	`{ muted, hours? }`	`{ muted, muted_until }`	No `hours` means indefinitely. Notifications only — unread still counts.
`POST …/unread`	—	`{ marked_unread }`	A flag of its own. The read watermark does not move, so their ticks stay exactly as they were.
`POST …/clear`	—	—	Hides every message for you. The thread stays in the list, empty; they keep everything.

Groups

A group is an ordinary conversation with more than two people in it. Sending, history, receipts, reactions, replies and every message action work unchanged — what differs is that it has a name and a picture, it is made rather than found, and it cannot be reduced to "the other person".

> HOW TO TELL THEM APART
>
> A conversation payload carries a `group` object when it is a group and `null` when it is not, and `other` is null in the opposite case. Branch on `group`, not on `type`.

The group object

```
{
  "group": {
    "title": "Saturday plans",
    "avatar_url": "https://…",   // signed, expiring, may be null
    "members_count": 4,
    "is_admin": true,           // whether *I* am an admin
    "created_by_me": true,

    // The minimum across everybody still in the room. Reported in the
    // same shape a direct thread reports one person, so the same tick
    // logic serves both: blue means the LAST person read it.
    "last_read_seq": 38,
    "last_delivered_seq": 40,

    // Only on the single-conversation view, null on the inbox.
    "members": [{ "id": "7a10…", "name": "Aisha", "avatar_url": "…" }],
    "roles": { "7a10…": "admin", "9f31…": "member" }
  }
}
```

> TWO IDS, TWO SPELLINGS
>
> A person inside `group.members` carries `id`; the `other` object on a direct thread carries `user_id`. They are the same uuid. Read both — this cost us a bug where every member parsed with an empty id and selecting one in a picker selected all of them.

`GET` `/conversations/group-candidates?scope=`

who may be added

SCOPE	RETURNS
`connections`	Everyone I follow, everyone who follows me, and my family. The default.
`family`	Family only — the "Family group" entry point.

```
curl -H 'Authorization: Bearer $TOKEN' \
  '…/conversations/group-candidates?scope=connections'

# → { scope, total, people: [{ id, name, username, avatar_url, … }] }
```

Blocked people are already excluded. This endpoint *offers* the list; it does not decide it — create enforces the same rule server-side, so a uuid that was not on this list is a 422 for the whole request.

POST `/conversations/group`

multipart · create

```
curl -X POST …/conversations/group \
  -H 'Authorization: Bearer $TOKEN' \
  -F 'title=Saturday plans' \
  -F 'scope=connections' \
  -F 'member_ids=["7a10…","9f31…"]' \
  -F 'avatar=@group.jpg'

# → 201, the full conversation object with group.members populated
```

FIELD	RULE
`title`	Required, 1–80 characters.
`member_ids`	Required, 1–99 uuids. Repeated fields or one JSON array — the server decodes a JSON string before validating.
`scope`	Which candidate list the ids must come from. Defaults to `connections`.
`avatar`	Optional image, ≤8 MB, checked by signature.

Without a picture, send it as plain JSON instead — same fields, `member_ids` as a real array. Everything arrives in one request either way, so a group is never created without the picture that was chosen for it.

The creator becomes the admin. A system message — `"Faisal created this group"` — is written immediately, because the inbox hides threads with no `last_message_at` and the group would otherwise be invisible until somebody spoke.

POST `/conversations/{uuid}/group`

rename · photo

DELETE `/conversations/{uuid}/members/{user}`

admin only

DELETE `/conversations/{uuid}`

leave

System messages

Group events arrive as ordinary messages with `type: "system"` and a plain-English `body`. They carry a `sender_id` — the person who did it — but should be drawn as a centred line rather than a bubble, and they never bump an unread count.

> WHERE THE TWO-PERSON ASSUMPTION STILL BITES
>
> Group message runs must be keyed on `sender_id`, not on "is it mine". Grouping consecutive messages the way a direct thread does will draw two different people's messages as one run under one name — somebody appearing to have said something they did not.

Uploads and media

Two steps, always: upload the bytes, then send a message that adopts them. A large file must not hold a chat request open, and the message row must not exist before its file does — a message broadcast ahead of its bytes gives every recipient a broken attachment and no way to recover.

`POST` `/uploads`

multipart · step one

```
curl -X POST …/uploads \
  -H 'Authorization: Bearer $TOKEN' \
  -F 'type=image' \
  -F 'file=@photo.jpg'

# voice notes carry their own measurements
curl -X POST …/uploads \
  -H 'Authorization: Bearer $TOKEN' \
  -F 'type=audio' \
  -F 'file=@note.m4a' \
  -F 'duration_ms=4200' \
  -F 'waveform=[12,40,88,61,…]'   # JSON, ≤128 ints 0–100

# → data.id — pass it as upload_id on the send
```

TYPE	ACCEPTS	MAX
`image`	jpg, png, webp, heic — checked by signature, not extension	8 MB
`file`	anything	25 MB
`audio`	AAC in an MP4 container (.m4a)	25 MB

> VOICE NOTES: THE CODEC IS NOT A PREFERENCE
>
> It must be **AAC-LC in .m4a**. Opus is smaller, but Safari cannot record it and iOS writes it into a container Android will not open — a note recorded in one browser would be silence in the other app. The server rejects anything else and names the type it saw. For the web recorder, that means `MediaRecorder` with `audio/mp4` where supported; Chrome only offers `audio/webm;codecs=opus`, so it needs a transcode step or a small WASM encoder before upload. Budget for this — it is the one genuinely awkward part of the web build.

An upload that is never sent is an orphan and is swept after 24 hours. Attachment URLs are **signed and expiring** — re-fetch the message rather than caching the URL, and never store it in your own database.

Presence

`POST` `/presence/ping`

I am here

Connecting to the socket

Reverb speaks the Pusher protocol, so in a browser this is `laravel-echo` + `pusher-js` pointed at our own host. The raw handshake is written out below because when a subscription is refused, this is the only place you can see why.

SETTING	VALUE
Host	`ws.sfamily.co`
Port	`443`, TLS
App key	`lrqwccbcdgoprday7bub` — public by design
Handshake URL	`wss://ws.sfamily.co/app/{key}?protocol=7&client=js&version=8.4`
Auth endpoint	`POST /api/v1/broadcasting/auth`, bearer token

> **NEVER SHIP THE SECRET**
>
> `REVERB_APP_SECRET` signs channel authorisations on the server and must never appear in a browser bundle, an env file served to the client, or a repo the frontend builds from. The *key* is public and identifies the app; the secret is not and does not.

The handshake, step by step

```
// 1. connect
← { "event": "pusher:connection_established",
     "data": "{\"socket_id\":\"812.3\",\"activity_timeout\":30}" }

// data arrives as a JSON *string*. Parse it twice. Echo hides this.

// 2. authorise the channel with the socket_id
→ POST /api/v1/broadcasting/auth
     Authorization: Bearer $TOKEN
     { "socket_id": "812.3",
       "channel_name": "private-conversation.9d2f…" }

← { "auth": "lrqwccbcdgoprday7bub:9f4c…" }
     // presence channels also return "channel_data"

// 3. subscribe
→ { "event": "pusher:subscribe",
     "data": { "channel": "private-conversation.9d2f…",
               "auth": "lrqwccbcdgoprday7bub:9f4c…" } }

← { "event": "pusher_internal:subscription_succeeded", … }

// 4. stay alive
← { "event": "pusher:ping" }
→ { "event": "pusher:pong", "data": {} }
```

> A 403 FROM THE AUTH ENDPOINT IS AN ANSWER
>
> It means a block, or a thread this account has left. Drop the channel — do not retry into a wall. A network *failure* is different: keep the channel wanted and retry on the next connect, or one flaky moment silently costs the user their live messages until they reload.

Channels

CHANNEL	SUBSCRIBE	CARRIES
`private-user.{your uuid}`	Sign-in to sign-out	`inbox.updated`, `conversation.closed`
`private-conversation.{uuid}`	While the thread is open	`message.sent`, `receipts.updated`, `message.reacted`, `conversation.pinned`
`presence-room.{uuid}`	While the thread is open	Who is looking, and the typing whisper

The mailbox channel is why the unread badge is right whether or not a chat is open. The conversation channel is subscribed only while its screen is open, so a message in another thread cannot reach a screen that is not showing it — that isolation is server-side, not a filter you can get wrong.

> WHY "ROOM" AND NOT "CONVERSATION"
>
> Laravel strips the `private-` and `presence-` prefixes before matching a channel definition, so a presence channel named `conversation` would collide with the private one. The presence channel is therefore `presence-room.{uuid}`.

Typing

A client event, whispered on the presence channel and relayed by Reverb without waking PHP. It never touches the database and is never queued — a fact that is worthless one second later has no business in a queue.

```
{ "event": "client-typing",
  "channel": "presence-room.9d2f…",
  "data": { "state": "typing",        // or "stopped"
            "user_id": "7a10…" } }  // who — required in a group
```

> SEND THE ID, RESOLVE THE NAME
>
> A whisper is relayed by Reverb without passing through the server, so anything in it is whatever the sending client chose to put there. Send the sender's **id** and look the name up in the member list the server sent — a name carried in the whisper itself would be taken on trust and could say anything. An id matching nobody is dropped: that is somebody who left between the keystroke and the frame.

Keep one expiry timer *per person*, not one for the room, or a client vanishing mid-sentence takes everybody else's indicator down with it. In a group, name the first and count the rest: `"Aisha is typing…"`, then `"Aisha +2 are typing…"` — three names across a header truncates to nothing useful.

Send `typing` at most every 2 seconds while keys are going in, and `stopped` on send or after ~3 seconds idle. Expire the indicator locally after ~6 seconds regardless — a client that disappears mid-sentence must not leave the other person watching a dot forever.

Events

Names come from `broadcastAs()`, so on the wire they are exactly as written below — no class path. In Echo they need a leading dot: `.listen('.message.sent', …)`.

`message.sent` · conversation channel

The full message object, identical to what the REST history returns.

```
{ "id": "3c81…", "client_id": "b0f2…", "seq": 42,
  "sender_id": "7a10…", "type": "text", "body": "…",
  "attachment": null, "reply_to": null, "reactions": [],
  "created_at": "2026-09-01T18:22:04+00:00" }
```

`receipts.updated` · conversation channel

```
{ "conversation_id": "9d2f…",
  "user_id": "7a10…",          // whose watermarks these are
  "last_read_seq": 42,
  "last_delivered_seq": 42,
  "unread_count": 0 }
```

Repaint every tick in the thread from these two numbers. Do not walk messages individually.

`message.reacted` · conversation channel

```
{ "message_id": "3c81…",
  "reactions": [{ "emoji": "❤️", "count": 2, "user_ids": ["…"] }] }
```

The complete set, not a delta. Replace, don't merge.

`conversation.pinned` · conversation channel

```
{ "conversation_id": "9d2f…",
  "pinned_message": { /* a message object */ } }   // null clears the banner
```

`inbox.updated` · user channel

The same conversation summary the inbox endpoint returns. Fold it into the list — this is the socket doing what a refresh would have done, without the refresh. Also where you report `delivered` from.

`conversation.closed` · user channel

```
{ "conversation_id": "9d2f…" }
```

A block or a departure. Remove the thread from the list and close the screen if it is open.

Using Echo in React

```
// npm i laravel-echo pusher-js
import Echo from 'laravel-echo';
import Pusher from 'pusher-js';

window.Pusher = Pusher;

export const echo = new Echo({
  broadcaster: 'reverb',
  key: 'lrqwccbcdgoprday7bub',
  wsHost: 'ws.sfamily.co',
  wsPort: 443,
  wssPort: 443,
  forceTLS: true,
  enabledTransports: ['ws', 'wss'],
  authEndpoint: 'https://admin-cp.sfamily.co/api/v1/broadcasting/auth',
  auth: { headers: { Authorization: `Bearer ${token}` } },
});

// the mailbox — subscribe once, at sign-in
echo.private(`user.${me.id}`)
  .listen('.inbox.updated', (thread) => store.absorb(thread))
  .listen('.conversation.closed', ({ conversation_id }) => store.remove(conversation_id));

// one thread — subscribe on mount, LEAVE on unmount
useEffect(() => {
  const channel = echo.private(`conversation.${id}`)
    .listen('.message.sent', onMessage)
    .listen('.receipts.updated', onReceipts)
    .listen('.message.reacted', onReaction)
    .listen('.conversation.pinned', onPinned);

  const room = echo.join(`room.${id}`)
    .here((members) => setPresent(members))
    .joining((m) => addPresent(m))
    .leaving((m) => removePresent(m))
    .listenForWhisper('typing', ({ state }) => setTyping(state === 'typing'));

  return () => { echo.leave(`conversation.${id}`); echo.leave(`room.${id}`); };
}, [id]);

// whisper while typing — throttle to one every 2s
room.whisper('typing', { state: 'typing' });
```

THREE THINGS THAT CATCH PEOPLE OUT

The **leading dot** on every custom event name. Calling `echo.leave()` — not just removing the listener — on unmount, or the next thread stacks a second subscription on top. And `listenForWhisper('typing')` without the `client-` prefix; Echo adds it for you.

Client rules that are not optional

1. **Order by `seq`, never by arrival or timestamp.** Insert into a sorted list; a message that overtakes another still lands in the right place.
2. **Deduplicate on `client_id`.** The sender receives their own broadcast. Key the optimistic bubble on the client uuid so the echo resolves to it rather than appearing twice.
3. **Fill the gap on reconnect.** On every socket re-establish, call `messages?after=<highest seq you hold>` for the open thread and refresh the inbox. The socket guarantees nothing about what happened while it was down.
4. **Report `delivered` from the inbox handler**, not the chat screen — otherwise the second grey tick only appears when the recipient opens the thread, which is what the blue one is for.
5. **Re-fetch attachment URLs.** They are signed and expire. Never persist one.
6. **Treat a 403 on send as a real state**, not an error toast: a block, or the fourth message to somebody who has not accepted the request. Show the composer's disabled state and say why.
7. **Retry with the same `client_uuid`.** A fresh one on retry is how duplicate messages happen.

Python scripts

Four small scripts that ship alongside this document. They are not part of the web app — they exist so the web team can prove the API and the socket behave as documented *before* React is in the way, and so a backend change that would break the client is caught in a place where the failure names itself.

`famzone_chat.py`

A reference client: every chat endpoint as a method, with the request shapes spelled out. Read it alongside the React service layer to settle arguments about what the API actually takes and returns. The other three import it.

`ws_listen.py`

Signs in, connects to Reverb, authorises and subscribes, then prints every frame with its payload. Run it in a second terminal while using the mobile app and watch the events arrive. This is the tool for "the socket isn't working" — it will show you a refused authorisation, a failed handshake, or a perfectly healthy connection carrying nothing because you subscribed to the wrong channel.

```
python ws_listen.py --phone 9876543210 --password secret123 \
                    --conversation <uuid> --typing
```

`smoke_test.py`

Signs in as two accounts and runs a whole conversation between them, asserting the behaviour the clients depend on: seq increments, a replayed `client_uuid` returns the same message, watermarks never regress, forward is capped at ten, delete-for-me removes it from one side and not the other. Every response is written to `./payloads/*.json`.

Those recorded payloads are the second reason to run it — point `quicktype` or `json-to-ts` at the folder and generate the TypeScript interfaces from real output rather than transcribing this page by hand. Regenerate when a field changes.

`seed_demo.py`

Fills a thread with a realistic history — long messages and one-word replies, quotes of older lines, reactions, a pinned message, and two unread on one side. A chat layout built against two messages of Lorem falls apart on contact with a real conversation; this is what you style against.

`group_demo.py`

Creates a group from whoever the account may add, then checks what the payload actually says about it: that it carries a `group` object and no `other`, that the creator is admin, that a system message opens the thread, that renaming works from any member, that removing drops the count and writes a line, and that you cannot remove yourself. Leaves the group at the end unless you pass `--keep`.

```
python group_demo.py --phone 9876543210 --password secret123
python group_demo.py --phone 9876543210 --password secret123 --scope family
```

> RUN AGAINST STAGING
>
> These send real messages to real accounts. `pip install requests websocket-client`, and set `FAMZONE_API` if you are not pointing at production.

Not built yet

So the web team plans around these rather than discovering them.

MISSING	WHAT IT MEANS FOR WEB
Push notifications	Nothing is delivered while the tab is closed. Web push is its own piece of work and is not specced yet.
Adding members to a group	A group's membership is fixed at creation. Removing works; adding does not exist yet. Admin-only when it lands.
Editing a message	`edited_at` is in the payload and always null. Do not build UI for it.
Message search	No endpoint. Client-side search over the loaded window only.
Report a message	Not built, and it is an App Store requirement for user content. Coming.
Per-member read info in a group	`/messages/{uuid}/info` answers for a direct thread. In a group it reports the room as a whole, not who read it when.
Favorites & custom lists	The WhatsApp-style list filters are not built.